Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras

IIT Madras

October 21, 2026
Git Revision: d76603b

All results reproducible from committed artifacts

1. Introduction
2. Related Work
3. Digital Twin Physics
4. Environment Design
5. Reinforcement Learning
6. Results
7. Interactive Workstation
8. Software Engineering
9. Discussion
10. Conclusion

CNC face milling: rotating cutter removes material in multiple passes.

Parameters to optimize:

- Spindle speed n [RPM]
- Feed per revolution f_r [mm/r]
- Axial depth of cut a_p [mm]

Four competing objectives:

- **Productivity:** Maximize $\mathrm{MRR} = v_c \cdot a_p \cdot f_r$
- **Energy:** Minimize total electrical energy/part
- **Quality:** $R_a \leq 1.6$ μm (finished surface)
- **Tool Life:** $V_B \leq 0.3$ mm, avoid breakage

Traditional fixed parameters (handbooks, static NSGA-II) cannot adapt to tool wear, material variation, or geometry changes.

1. **Digital Twin (Physics)**
   - Speed-dependent cutting energy $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$
   - Extended Taylor tool wear
   - Empirical roughness regression
   - Stochastic breakage
2. **DRL Agent (PPO)**
   - One Gymnasium environment
   - 11-D observation, context always present
   - Energy-aware reward
   - Conditioned policy (w_2 + material context)
3. **Experimental Rigor**
   - 100-scenario Monte Carlo + 95% CIs
   - Ablation: 7 weights $\times$ 3 seeds
   - 5 classical baselines + receding-horizon
   - Transfer experiment
   - Full reproducibility
4. **Interactive Workstation**
   - 2D stock depletion, micro-topography
   - G-code generation, multi-controller comparison

Capability	Mo et al. (2025) *J. Manuf. Process 156*	Wang et al. (2025a) *J. Manuf. Syst. 78*	**This Work**
Dynamic optimization	✓	✓	✓
Tool wear in the state	✓	✗	✓
Roughness constraint	✓	✗	✓
Energy as an objective	✗	✗	✓
Speed-dependent k_c	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy	✗	✗	✓
Transfer experiment	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Monte Carlo (100 scenarios)	✗	✗	✓
Interactive workstation	✗	✗	✓

Mo *et al.*: PPO over a mixed action space (tool change + parameters); the state carries V_B, breakage and R_a; objectives are quality and efficiency.
Wang *et al.*: MATD3 for batch milling; objectives are dimensional quality and machining time.

Key differentiator: **neither reference treats energy as an objective**. The speed-dependent k_c creates a genuine energy/throughput trade-off, and one conditioned policy covers the whole frontier in a single run.

$$v_c = \frac{\pi D n}{1000} \tag{1}$$

$$\mathrm{MRR} = v_c \cdot a_p \cdot f_r \tag{2}$$

$$t_{\text{cut}} = \frac{L_{\text{cut}}}{n f_r} \tag{3}$$

$$\Delta V_B = C_{\text{wear}} v_c^{\alpha} f_r^{\beta} a_p^{\gamma} t_{\text{cut}} \tag{4}$$

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{5}$$

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^m \tag{6}$$

$$P_{\text{total}} = P_{\text{idle}} + k_c(v_c) \cdot \mathrm{MRR} \tag{7}$$

$$E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}} \tag{8}$$

Key Insight: Interior Minimum

With $m = 0.6 > 0$:

- Idle energy $\propto 1/(n f_r) \downarrow$ with speed
- Cutting energy $\propto n^m \uparrow$ with speed
- Part energy $E_{\text{part}}(v_c)$ has **interior minimum**
- Energy weight w_2 selects operating point

Validated by a unit test on the part energy minimum

One registration

- `env.ENV_ID` = `MillingDTEnv-v1`, 200-step limit
- The energy weight and the material are *inputs*, not separate tasks

Observation (11-D, always the same shape)

- 0–7: tanh-squashed V_B, κ_{rem}, previous n, f_r, a_p, $\mathrm{MRR}/\mathrm{MRR}_{\max}$, tanh-squashed R_a, tool-broken flag
- 8–10: the context: w_2, C_{wear} over $[0.6, 1.4]$, k_1 over $[0.8, 1.2]$ of nominal

Action: $[-1, 1]^3 \to$ clipped $\to$ machine envelope

Reward per step

$$r = 5\left(w_1 \frac{\mathrm{MRR}}{\mathrm{MRR}_{\max}} - w_2 \frac{E_{\text{pass}}}{E_{\text{ref}}}\right) + 2\,\Delta\left(\frac{\kappa_{\text{removed}}}{\kappa_{\text{total}}}\right) \quad (9)$$

$$- 50\,\mathbb{1}[V_B > V_{B,\text{safe}}] - 100\,\mathbb{1}[\text{broken}] \quad (10)$$

$$- 50\,\mathbb{1}[\text{finish} \wedge R_a > R_{a,\text{safe}}] - 0.05 \quad (11)$$

- $+200$ when a part is finished in spec, plus a Pareto margin bonus of at most $+5$
- $w_1 = 1 - w_2$; safety limits $V_{B,\text{safe}} = 0.255$ mm and $R_{a,\text{safe}} = 1.36$ μm
- breakage $p = 0.3 \cdot$ wear factor $\cdot$ (load factor)^2 above $V_B > 0.15$ mm
- the objective scale of 5 lets w_2 compete with the shaping terms

- **Workpiece:** $\kappa_{\text{total}} \in [2, 15]$, $D \in [40, 150]$, $L_{\text{cut}} \in [50, 200]$
- **Material/Tool:** C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 at $[0.8, 1.2]\times$ nominal
- **Initial wear:** $V_{B,0} \sim \mathcal{U}[0, 0.8]V_{B,\max}$
- **Energy weight:** drawn $\mathcal{U}[0, 1]$ per episode under `-randomize-weight`
- **Shared tables:** same `MillingDT/config/scenarios.py` for training + Monte Carlo eval
-
- **Measured:** the normalized physics features the policy sees on the training and the evaluation sets agree to 0.041 in the mean (std = 1), the same distribution
- **In the observation:** C_{wear} and k_1 travel with the part, so the policy knows which material it is cutting

Table: Hyperparameters

Parameter	Value
Policy	MlpPolicy (2×256)
Learning rate	3×10^{-4} (linear)
n_steps	2048
Batch size	64
n_epochs	10
γ / GAE λ	0.99 / 0.95
Clip range	0.2
Entropy coeff	0.02
VecNormalize	norm_obs=True, norm_reward=True
Envs / Device	4 (`SubprocVecEnv`) / CPU

Training Runs

- **Production policy:** 1M steps, seed 42, `-randomize -randomize-weight` → `results/run_PPO_seed42/`
- **Ablation grid:** 7 w_2 × 3 seeds = 21 policies, 150k steps each

Every ablation policy has its energy weight pinned at training and is evaluated at that same weight, so a policy is always asked the question it was trained for. The production policy instead sees the weight drawn per episode.

Heartbeat callback every 1000 steps → stall detection in `run_experiments.py`

Metric	Mean	Std	95% CI
MRR (mm^3/min)	1205.2	585.7	[1088.9, 1321.4]
Energy (kJ)	68.4	48.3	[58.8, 77.9]
Cycle time (s)	20.3	17.1	[16.9, 23.6]
Max R_a (μm)	1.379	0.350	[1.309, 1.448]
Max V_B (mm)	0.1244	0.0679	[0.1109, 0.1378]

Constraint	Rate	95% CI
R_a viol., worst pass	38.0%	[28.5, 47.5]%
R_a viol., finished surface	0.0%	[0, 0]%
R_a above safety limit, worst	56.0%	[46.3, 65.7]%
R_a above safety limit, finished	0.0%	[0, 0]%
$V_B > 0.3$ mm	0.0%	[0, 0]%
Tool breakage	1.0%	[0, 3]%
Parts completed in spec	99.0%	

Key Findings

- Finished surface always in spec (0% violations), the part quality holds
- Wear never exceeded (0% violations), 99% of the parts completed
- 56% of the passes exceed the safety margin $R_{a,\text{safe}} = 1.36$ μm: the policy pushes and recovers by the finish pass
- Mean operating point: 7199 RPM, 0.247 mm/r, 2.11 mm, 4.0 passes

Table: Ablation: mean ± std over 3 seeds, 50 scenarios each, weight pinned

w_2	MRR	Energy (kJ)	Cycle (s)	n (RPM)	f_r	Frontier
0.0	946.3±63.4	82.9±1.4	44.8	7094	0.166	dominated
0.2	1007.8±22.7	77.1±1.1	31.3	7471	0.184	dominated
0.4	1038.0±67.6	72.7±0.5	24.8	7256	0.203	✓
0.5	1035.8±43.8	70.3±1.2	22.0	7080	0.214	✓
0.6	1023.9±27.4	70.8±0.6	22.1	7148	0.209	dominated
0.8	971.1±43.2	66.8±1.2	21.9	6538	0.228	✓
1.0	951.3±54.7	65.8±0.6	20.9	6490	0.229	✓

Frontier Analysis

From $w_2 = 0.4$ to $w_2 = 1.0$: the removal rate falls 8.4% (1038 → 951 mm^3/min) and the energy falls 9.5% (72.7 → 65.8 kJ). The trade-off is **symmetric**. The spindle speed falls with the energy weight (7256 → 6490 RPM): the energy is bought with throughput, not with time. Dominated points ($w_2 = 0.0, 0.2, 0.6$) remove less for more.

w_2	MRR, one policy	MRR, per weight	MRR gap	Energy, one policy	Energy, per weight	Energy gap
0.0	1213.6	946.3	+28.2%	69.6	82.9	−16.0%
0.2	1215.3	1007.8	+20.6%	69.0	77.1	−10.5%
0.4	1205.1	1038.0	+16.1%	68.3	72.7	−6.0%
0.5	1191.0	1035.8	+15.0%	68.0	70.3	−3.3%
0.6	1169.3	1023.9	+14.2%	67.6	70.8	−4.5%
0.8	1130.8	971.1	+16.4%	67.1	66.8	+0.5%
1.0	1096.0	951.3	+15.2%	67.1	65.8	+2.1%

Result

For w_2 from 0.0 to 0.6 the single policy removes 14% to 28% more material for 3% to 16% less energy. At $w_2 = 0.8$ and $w_2 = 1.0$ it still removes 16% and 15% more for 0.5% and 2.1% more energy.

The same policy swept at runtime: MRR 1213.6 ($w_2 = 0.0$) → 1096.0 ($w_2 = 1.0$), energy 69.6 → 67.1 kJ, spindle 7513 → 6475 RPM. **The weight stays a runtime query**, not a retrain: the observation carries w_2 and the material context (C_{wear}, k_1).

Table: Baseline comparison on the same 50 episodes

Policy	Reward	MRR	Energy (kJ)	Cycle (s)
PPO (ours)	204.09	894.3	18.40	5.56
Receding-Horizon	203.45	**981.7**	**18.02**	**4.80**
NSGA-II Static	-46.39	732.9	18.8	6.4
Rule-Based	-46.48	719.8	18.7	6.6
Constant-High	-46.10	1099.6	17.8	4.3 (violates)
Constant-Safe	-139.25	1.6	1506.8	3000.0

- **Same 50 episodes** of the nominal condition for all six policies, at $w_2 = 0.4$
- **Level on the reward:** PPO 204.1 against the controller 203.5
- **The controller is ahead** on the removal rate (981.7 against 894.3) and the cycle time (4.80 s against 5.56 s)
- No reinforcement learning advantage is demonstrated yet
- The three fast static policies finish outside the roughness safety limit, so the reward takes the -50 finishing penalty; Constant-Safe stays inside but needs 3000 s

Question: does a policy that starts from the controller end up better than the expert it started from?

Method

1. Record the controller on 2000 randomized episodes
2. Fit a supervised actor (2×256, Tanh) on those trajectories by regression
3. Copy the fitted weights into a fresh PPO policy
4. Train with the production budget (1M steps), evaluate on 100 scenarios at $w_2 = 0.4$

`MillingDT/agent/controller_init.py` `experiments/controller_experiment.py`

Metric	Receding-horizon controller	PPO from scratch	PPO initialized from controller
Mean episode reward	187.96	198.57	199.35
Reward std	65.92	30.03	3.35
MRR (mm^3/min)	1560.6	1205.1	1126.1
Energy per part (kJ)	68.01	68.35	77.17
Cycle time (s)	13.54	20.25	32.35
Tool breakage	5%	1%	0%
Parts completed	95%	99%	100%

No advantage over the controller

- The controller still removes 38% more material per minute (1560.6 against 1126.1 mm^3/min)
- and finishes in 13.5 s against 32.3 s
- The reward cannot separate the two learned policies: 199.35 against 198.57 is a gap of 0.8, while the standard error of the mean over 100 scenarios is $30.0/\sqrt{100} = 3.0$
- All three arms keep the finished surface in spec (0% violations)

Consistency, paid for with throughput and energy

- Reward std across the 100 scenarios falls from 30.0 to 3.4, tool breakage falls to zero, every part is completed
- Against the from-scratch policy: 6.6% less removal rate and 12.9% more energy per part
- The controller is the least reliable arm of the three: std 65.9, 5% breakage, 95% of the parts completed

What the work claims

The reinforcement learning agent navigates the energy/throughput trade-off inside a fixed constraint set. It does not beat a classical controller that re-optimizes every pass with the exact physics.

Target material (harder alloy, same machine): $k_1 \times 1.25$, $k_2 \times 1.30$, $C_{\text{wear}} \times 2.0$. The source policy was trained for 300,000 steps on the nominal material; the adaptation budget on the target is 50,000 steps.

Arm	**Reward**	R_a **viol.** worst pass	R_a **viol.** finished surface	**Parts** completed
Source policy, zero shot	182.5	100.0%	3.3%	93.3%
Source policy, fine tuned (50k)	196.6	0.0%	0.0%	100.0%
Trained on the target from scratch (50k)	196.6	0.0%	0.0%	100.0%
Production policy, zero shot	**200.3**	60.0%	0.0%	100.0%

How to read this

The production policy was trained across a **distribution of materials** with the material context in its observation, so the harder material is inside its training distribution and it needs no adaptation at all. It scores the highest reward (200.3) and the finished surface is in spec in every scenario; its 60% worst-pass violations fall on the roughing passes, which the reward does not constrain: the finished surface, which is the part quality, is in spec everywhere. The nominal-trained source policy cannot do this: 182.5, and it violates the roughness limit on the worst pass in every scenario. Fine tuning it for 50,000 steps, or training on the target from scratch, both fix it, and both land at 196.6.

- **2D Stock Depletion:** Cylindrical billet cross-section; pass-by-pass cutout; remaining κ, a_p; tool engagement
- **Micro-Topography:** Feed-mark cusp $y_{\text{cusp}}(x) \approx x^2/(8r_\epsilon) + \text{noise}$; ISO 1302 R_a overlay; 1.6 μm ceiling
- **G-Code Generator:** ISO 6983 Fanuc/Siemens compliant; exact PPO sequence; tool/diameter aware
- **Multi-Controller:** Side-by-side PPO vs NSGA-II vs Rule-Based vs Constant-High; live telemetry
- **Cached Loading:** `@st.cache_resource` loads model once per session

```
streamlit run app.py
```

Module Structure

- `MillingDT/`: config, env, agent (controller init), baselines, sim, utils
- `experiments/`: train, eval, ablation, baselines, transfer, controller, pipeline
- `viz/`: shared plotting utilities
- `results/`: one directory per run (models, log, metrics, plots)

Reproducibility

- Global seeding (Python, NumPy, PyTorch, CUDA)
- Shared scenario tables (train = eval)
- `experiments/summarize_results.py` regenerates `RESULTS.md`

Test Coverage (34 tests)

- 22 env tests: clipping, truncation, soft norm, 11-D observation, constants, overrides, energy norm, Gym checker, randomization, pinned values, scenario scales, context (weight, material), speed dependence, energy minimum, shared physics
- 2 reward tests: golden values ($w_2 = 0.0, 0.4, 0.75$), finishing penalty
- 5 integration: train$\rightarrow$eval roundtrip, one registration, context in the observation, one policy at a pinned weight, baseline artifact
- 3 callback: heartbeat, failure report, interval
- 2 training: convergence, reproducibility

Code Quality

- No unused imports (AST verified)
- Constants and paths in `MillingDT/config/`

What Worked

- Speed-dependent k_c creates genuine trade-off
- Domain randomization $\rightarrow$ 99% completion, 0% finished violations
- One policy beats the 21 per-weight policies
- Training across materials makes adaptation unnecessary
- Receding-horizon = strong, fair baseline
- Controller initialization buys consistency: reward std 30.0 $\rightarrow$ 3.4, no breakage

Limitations

- PPO is level with receding-horizon, not ahead of it
- Initializing from the controller does not close the gap: more uniform, but 6.6% less removal rate and 12.9% more energy
- Ablation: 150k steps/weight (vs 1M production)
- Reward charges a pass, metric is a part
- Sim-only, no real calibration yet

Future Work

- Compare against a restricted controller (is the gap an information gap?)
- Sim-to-real: calibrate, add noise/delay
- True multi-objective RL (Pareto, hypervolume)
- Online w_2 adaptation for energy budget

- **Digital Twin** with speed-dependent cutting energy, tool wear, roughness, stochastic breakage
- **One Gymnasium Environment** (`MillingDTEnv-v1`) with domain randomization and an 11-D observation that always carries the context
- **One Production Policy** (1M steps): the energy weight is a runtime query, not a retrain
- **Rigorous Evaluation**: Monte Carlo (100 scenarios, 95% CIs), ablation frontier, weight sweep against the 21 per-weight policies
- **Baselines**: six policies on the same 50 episodes, level on the reward, so no RL advantage yet
- **Controller Initialization**: more uniform (reward std 3.4 against 30.0), no tool breakage, but the controller still removes 38% more material per minute
- **Transfer Experiment**: the production policy needs no adaptation; the nominal-trained one does
- **Interactive Workstation**: 2D stock depletion, micro-topography, G-code, multi-controller comparison
- **Full Reproducibility**: every result table from committed artifacts by one script

Key Scientific Finding

The speed-dependent specific cutting energy creates a **genuine energy/throughput trade-off**, and DRL learns to navigate it pass-by-pass while respecting surface quality and tool life constraints. Across the measured frontier the removal rate falls 8.4% while the energy of the part falls 9.5%. One conditioned policy reaches that whole frontier at runtime, beats the 21 policies trained per weight, and transfers to a harder material with no adaptation.

Task	Command
Tests	`pytest tests/ -v`
Production	`python -m experiments.train_rl -steps 1000000 -seed 42 -randomize -randomize-weight`
Pinned weight	`python -m experiments.train_rl -steps 150000 -seed 42 -randomize -w2 0.6`
Ablation grid	`python -m experiments.train_rl_ablation -steps 150000 -scenarios 50 -slots 7`
MC eval	`python -m experiments.evaluate -scenarios 100`
Weight sweep	`python -m experiments.evaluate -sweep-weight -scenarios 100`
Compare	`python -m experiments.evaluate -compare-only`
Baselines	`python -m experiments.run_baselines`
Transfer	`python -m experiments.transfer_experiment 300000 50000`
Controller init	`python -m experiments.controller_experiment 1000000 2000`
Results	`python -m experiments.summarize_results`
Workstation	`streamlit run app.py`
Full pipeline	`python -m experiments.run_pipeline -steps 1000000`

Thank You

Questions & Discussion

Repository: `G:/My Drive/IITM/AI in Mfg`
ME6324 – AI in Manufacturing | IIT Madras